

Labo 04 — Le Dockerfile : construire ses propres images

Théorie — la recette, le contexte, le cache, et les deux pièges qui coûtent le plus cher.

Objectifs

- Comprendre ce qu'est le **build context** et pourquoi il détermine ce que vous pouvez copier.
- Connaître les instructions essentielles et ce que chacune produit.
- Distinguer `CMD` et `ENTRYPOINT`, forme *shell* et forme *exec*.
- Distinguer `ARG` et `ENV`.
- Ordonner un Dockerfile pour exploiter le **cache de construction**.

1. Le build context

```
docker build -t mon-api:1.0 .
```

Le `.` final n'est pas décoratif : c'est le **contexte de construction**. Le client `docker` empaquette ce dossier — récursivement — et l'envoie au daemon. Deux conséquences absolues :

- **Vous ne pouvez copier que ce qui est dans le contexte.** `COPY ../secrets/cle.pem .` échoue toujours : le fichier n'a jamais été envoyé. Il n'y a pas de contournement.
- **Tout ce qui est dans le dossier est transféré**, y compris `.git/`, `node_modules/`, `target/`, les logs et les fichiers de configuration locaux. Sur un projet Angular, cela peut représenter des centaines de Mo à chaque build.

Le fichier `.dockerignore`, à la racine du contexte, exclut ce qui ne doit pas partir. C'est un fichier d'hygiène de base :

```
.git
node_modules
target
*.log
.env
**/application-local.yml
```

Piège — Sans `.dockerignore`, un `COPY . .` embarque les secrets locaux, les identifiants et le `.git` complet **dans l'image finale**, où ils resteront pour quiconque la télécharge. C'est une fuite de données classique.

Le Dockerfile lui-même n'a pas besoin d'être dans le contexte : `-f` le désigne (`docker build -f docker/api.Dockerfile -t api:1.0 .`).

2. Les instructions essentielles

```
FROM eclipse-temurin:21-jre-alpine      # image de base – toujours la 1re instruction
LABEL org.opencontainers.image.source="https://git.masociete.fr/paiement/api"
WORKDIR /app                            # crée et se place dans le dossier
COPY target/api.jar /app/api.jar        # copie depuis le contexte vers l'image
ENV JAVA_OPTS="-XX:MaxRAMPercentage=75" # variable présente à l'exécution
EXPOSE 8080                             # documentation : ne publie AUCUN port
USER 1000:1000                           # ne pas tourner en root
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar /app/api.jar"]
```

Instruction	Rôle	Couche de fichiers ?
FROM	Point de départ	oui (celles de la base)
RUN	Exécute une commande au build	oui
COPY / ADD	Copie depuis le contexte	oui
WORKDIR, ENV, USER, EXPOSE, LABEL	Métadonnées	non (0B)
CMD, ENTRYPOINT	Ce qui s'exécute au <code>docker run</code>	non
ARG	Variable du build seulement	non

Trois précisions utiles. **COPY plutôt qu'ADD** : `ADD` décompresse automatiquement les archives locales et sait télécharger une URL, deux comportements implicites et surprenants ; réservez-le à l'extraction d'une archive locale. **EXPOSE ne publie rien** : c'est une déclaration d'intention, la publication réelle se fait avec `-p` (labo 07). **RUN s'exécute au build, jamais à l'exécution** : `RUN java -jar api.jar` lancerait votre application pendant la construction de l'image — confusion fréquente.

3. CMD contre ENTRYPOINT

Les deux définissent ce qui tourne au démarrage. Leur différence est leur rapport aux arguments de `docker run` :

- CMD** est une valeur **par défaut, remplaçable**. `docker run mon-image autre-commande` ignore le `CMD` et exécute `autre-commande`.
- ENTRYPOINT** est le programme **fixe**. Les arguments de `docker run` lui sont **ajoutés**.

```
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

`docker run api` lance `java -jar /app/api.jar --spring.profiles.active=prod`. `docker run api --spring.profiles.active=dev` lance la même chose avec le profil `dev`. C'est le motif standard : `ENTRYPOINT` fixe le programme, `CMD` fournit les arguments par défaut.

En pratique : `ENTRYPOINT` (+ `CMD`) pour une image d'application ; `CMD` seul pour une image utilitaire dont on veut pouvoir remplacer la commande. Dans le premier cas, `docker run --entrypoint sh -it mon-image` reste la porte de sortie pour déboguer.

4. Forme *shell* et forme *exec*

Toute commande peut s'écrire de deux façons, et ce n'est pas une question de style :

```
CMD java -jar /app/api.jar # forme SHELL -> /bin/sh -c "java -jar ..."  
CMD ["java","-jar","/app/api.jar"] # forme EXEC -> java devient PID 1
```

En forme *exec*, l'application **est** le PID 1 : elle reçoit `SIGTERM` et peut s'arrêter proprement. En forme *shell*, un `/bin/sh` s'intercale — et c'est là que se joue le problème du labo 03 : un shell qui reste PID 1 ne relaie pas `SIGTERM`, donc l'application ne le reçoit jamais, `docker stop` attend dix secondes et tue tout brutalement.

Le mot important est **reste**. Le comportement dépend de l'implémentation du shell :

Cas	PID 1	docker stop
Forme <i>exec</i>	votre application	propre, code 143
Forme <i>shell</i> , commande simple, base Alpine (busybox)	votre application (le shell s'efface)	propre, code 143
Forme <i>shell</i> , commande simple, base Debian/Ubuntu (dash)	<code>/bin/sh</code>	10 s puis code 137
Forme <i>shell</i> avec un tube, un <code>&</code> , un <code>;</code>	<code>/bin/sh</code>	10 s puis code 137
Script d'entrée qui lance l'appli sans <code>exec</code>	<code>/bin/sh</code>	10 s puis code 137

Autrement dit, la forme *shell* est parfois inoffensive et parfois destructrice, selon l'image de base et la commande — ce qui est bien pire qu'un comportement franchement mauvais : cela marche sur le poste du développeur (Alpine) et casse en production (Debian).

À retenir — Écrivez toujours la forme *exec*, avec des guillemets doubles JSON. Si vous devez passer par un shell, écrivez-le explicitement **et** utilisez `exec` : `ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/api.jar"]`.

Deux conséquences moins connues : en forme *exec*, `$JAVA_OPTS` n'est **pas** interprété (il n'y a pas de shell pour le faire), et `&&`, `|`, `>` non plus.

5. ARG contre ENV

```
ARG VERSION=1.0 # disponible pendant le build uniquement  
ENV APP_VERSION=${VERSION} # persiste dans l'image et dans les conteneurs
```

	ARG	ENV
Visible pendant le build	oui	oui
Présent dans l'image finale	non	oui
Modifiable au build	<code>--build-arg VERSION=2.0</code>	non
Modifiable à l'exécution	non	<code>docker run -e APP_VERSION=...</code>

Piège — « ARG disparaît de l'image » ne veut **pas** dire « ARG est sûr pour un secret ». La valeur reste visible dans `docker history` et dans le cache de build. Un mot de passe passé en `--build-arg` est une fuite. Voir le labo 08.

6. Le cache de construction

Docker traite les instructions dans l'ordre et met chaque résultat en cache. Pour chaque instruction, il se demande : « ai-je déjà exécuté celle-ci, à partir de la même couche précédente ? » Si oui, il réutilise (`CACHED`). Sinon il l'exécute — **et invalide tout ce qui suit**.

L'invalidation est déclenchée par :

- une modification du texte de l'instruction ;
- pour `COPY / ADD`, une modification du **contenu** des fichiers copiés ;
- l'invalidation d'une instruction précédente, en cascade.

D'où la règle d'or : **du plus stable au plus volatil**.

```
# MAUVAIS : le code change à chaque commit, donc tout est reconstruit
COPY . /app
RUN mvn dependency:go-offline

# BON : les dépendances ne sont retéléchargées que si pom.xml change
COPY pom.xml /app/
RUN mvn dependency:go-offline
COPY src /app/src
```

Le même raisonnement s'applique à Angular : `COPY package*.json` puis `npm ci`, et seulement ensuite `COPY . .`. Le gain se compte en minutes à chaque build de CI — et, puisqu'une couche inchangée n'est pas retransférée au `push` (labo 02), en temps de déploiement. Deux options utiles au diagnostic : `--no-cache` pour tout reconstruire, et `--progress=plain` pour voir la sortie complète de chaque étape.

7. Écrire un `RUN` correct

```
# MAUVAIS : trois couches, un cache d'apt de 40 Mo embarqué dans l'image
RUN apt-get update
RUN apt-get install -y curl
RUN rm -rf /var/lib/apt/lists/*

# BON : une seule couche, nettoyage effectif
RUN apt-get update \
&& apt-get install -y --no-install-recommends curl \
&& rm -rf /var/lib/apt/lists/*
```

Deux règles en découlent. Le nettoyage doit avoir lieu **dans le même `RUN`** que l'installation : une suppression dans une couche ultérieure masque les fichiers sans les retirer de l'image (labo 02). Et `apt-get update` ne doit jamais être seul dans sa couche : mis en cache pendant des semaines, il servirait des index périmés à l'installation suivante — c'est le problème dit du *cache busting*.

8. En entreprise

Le Dockerfile d'un back Spring Boot ressemble à ceci (version simple ; la version multi-stage arrive au labo 05) :

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/api-*.jar app.jar
EXPOSE 8080
USER 1000:1000
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Points à noter : image de base **JRE** et non JDK, `USER` non-root, forme exec, JAR copié depuis `target/` — donc le build Maven a eu lieu **avant**, sur la machine ou en CI. C'est précisément la limite que le multi-stage va lever.

Côté Angular, le principe est le même : on ne conteneurise jamais le serveur de développement `ng serve`, mais le résultat de `ng build` servi par nginx.

À retenir

- Le `.` de `docker build .` désigne le **contexte** : tout ce qui est copiable, et tout ce qui est transféré. `.dockerignore` est obligatoire, pas optionnel.
- `EXPOSE` documente, `-p` publie. `RUN` s'exécute au build, `CMD` / `ENTRYPOINT` à l'exécution.
- `ENTRYPOINT` fixe le programme, `CMD` fournit des arguments remplaçables.
- Forme exec `["prog", "arg"]` : l'application est PID 1 et reçoit `SIGTERM`. Forme *shell* : elle ne le reçoit pas.
- `ARG` vit pendant le build, `ENV` persiste dans l'image — aucun des deux ne convient à un secret.
- Ordonnez du plus stable au plus volatil ; toute instruction invalidée invalide les suivantes.
- Installation et nettoyage dans le **même** `RUN`, sinon les fichiers restent dans l'image.

Vocabulaire

build context : dossier envoyé au daemon lors du build. — `.dockerignore` : liste des exclusions du contexte. — **forme exec** / **forme shell** : deux écritures de `CMD` / `ENTRYPOINT`. — **cache de build** : réutilisation des couches déjà construites. — **cache busting** : invalidation volontaire du cache pour rafraîchir des données. — **build arg** : variable de build (`--build-arg`). — **image de base** : image citée par `FROM`.